PhysioHome App Documentation

Product, Functional, Technical, and Operational Documentation

Document Type: Product and Engineering Documentation
Product Name: PhysioHome
Product Status: Concept / Planning / MVP Definition
Prepared For: Product, Design, Engineering, QA, Operations, and Stakeholder Review
Brand Guideline: To be documented separately
Primary Market: Kenya, with future expansion across Africa
Platform Type: Mobile-first healthcare service marketplace
Core User Groups: Patients, Physiotherapists, Administrators

---

1. Document Purpose

This document defines the product structure, functional scope, system behavior, user flows, technical architecture, data model, backend modules, operational rules, security expectations, testing strategy, and future roadmap for the PhysioHome application.

The purpose is to provide a professional reference that can guide product planning, UI/UX design, frontend development, backend development, quality assurance, stakeholder discussions, and future scaling decisions.

This document does not define the final brand identity, logo rules, visual identity, brand voice, color psychology, typography system, or marketing assets. Those will be handled in a separate brand guideline document.

---

2. Executive Summary

PhysioHome is a mobile-first healthcare service marketplace designed to connect patients with licensed physiotherapists for home-based physiotherapy services. The platform allows patients to request physiotherapy care from their homes, compare available professionals, schedule sessions, make payments, track booking progress, and submit reviews after treatment.

For physiotherapists, the platform provides a professional portal for profile management, availability control, request handling, service delivery tracking, earnings visibility, and verification status management.

For administrators, the system supports professional verification, user oversight, payment monitoring, dispute handling, booking supervision, analytics, and operational control.

The core value of PhysioHome is accessibility. Many patients who need physiotherapy face mobility challenges, transport barriers, long hospital queues, limited specialist access, and repeated travel costs. PhysioHome reduces these barriers by bringing verified physiotherapy care closer to the patient.

---

3. Product Vision

PhysioHome aims to become a trusted healthcare access platform for home-based physiotherapy and rehabilitation services.

The long-term vision is to build a reliable digital infrastructure where patients can safely access qualified physiotherapists, physiotherapists can grow their practice, and healthcare teams can coordinate rehabilitation care more efficiently.

The product should feel safe, calm, professional, accessible, and trustworthy. Since the target users include patients with pain, disability, limited mobility, age-related challenges, or post-surgical needs, the product must prioritize clarity, simplicity, and confidence over unnecessary complexity.

---

4. Problem Statement

Accessing physiotherapy services can be difficult for patients who are elderly, injured, recovering from surgery, managing chronic pain, recovering from stroke, or unable to travel easily. In many cases, patients delay rehabilitation because they cannot find trusted specialists nearby, cannot travel comfortably, or cannot verify whether a provider is properly qualified.

Physiotherapists also face challenges. Many independent professionals struggle to reach patients beyond referrals, manage appointments efficiently, communicate availability, handle payments, and build digital trust.

PhysioHome addresses these problems by providing a structured marketplace where verified physiotherapists can be matched with patients based on location, specialty, availability, rating, and service type.

---

5. Product Objectives

The primary objectives of PhysioHome are to simplify access to physiotherapy services, increase trust in home-based care, support verified physiotherapist discovery, reduce patient friction during booking, enable secure payment flows, and create an operational structure that can scale as demand grows.

The application must support both scheduled and urgent care requests, while ensuring that only verified physiotherapists can accept patient bookings. It must also provide administrators with enough control to maintain quality, safety, and compliance.

---

6. Product Scope

6.1 MVP Scope

The minimum viable product should include patient registration, physiotherapist registration, role-based authentication, patient profile setup, physiotherapist profile setup, physiotherapist verification submission, admin approval, service category selection, location-based physiotherapist listing, booking creation, booking acceptance or rejection, booking status tracking, M-Pesa payment initiation, payment confirmation, push notifications, session completion, rating submission, and basic admin oversight.

6.2 Post-MVP Scope

Post-MVP features may include AI symptom guidance, telehealth consultations, rehabilitation progress tracking, multi-session packages, caregiver accounts, insurance integrations, subscription plans, automated follow-up reminders, advanced provider ranking, referral management, in-app chat, video consultations, and corporate wellness partnerships.

6.3 Out of Scope for MVP

The MVP should not attempt to include full hospital EMR integration, insurance claim automation, video consultations, wearable device integration, advanced AI diagnosis, prescription handling, or complex multi-branch healthcare facility management unless specifically prioritized later.

---

7. Target Users

7.1 Patients

Patients are individuals who need physiotherapy services at home. They may be recovering from surgery, managing back pain, recovering from stroke, dealing with sports injuries, supporting elderly mobility, managing musculoskeletal conditions, or seeking general rehabilitation support.

The patient experience must be simple enough for non-technical users. The app should avoid unnecessary medical jargon, provide clear actions, and make booking feel safe.

7.2 Physiotherapists

Physiotherapists are licensed professionals who provide home-based physiotherapy services. They need tools to create a professional profile, submit verification documents, define specialties, manage availability, receive bookings, accept or reject requests, track appointments, complete sessions, and monitor earnings.

The physiotherapist experience must support professional credibility and operational efficiency.

7.3 Administrators

Administrators are internal users responsible for managing the platform. They verify physiotherapists, monitor bookings, resolve disputes, manage users, oversee payments, review flagged activity, and support customers.

The admin experience must provide visibility, control, and auditability.

---

8. User Roles and Permissions

8.1 Patient Role

Patients can register, log in, manage their profile, select a service, choose location, request a booking, pay for a booking, track booking status, cancel eligible bookings, view history, rate physiotherapists, and receive notifications.

Patients cannot approve physiotherapists, access other patients' data, view provider documents, change payment status manually, or modify system configuration.

8.2 Physiotherapist Role

Physiotherapists can register, submit professional verification details, manage their profile, set availability, view assigned or available requests, accept or reject bookings, update session progress, mark sessions as completed, view earnings, and receive patient reviews.

Physiotherapists cannot accept bookings before approval, view unrelated patient records, manually override payment status, approve other physiotherapists, or access admin analytics.

8.3 Admin Role

Admins can view users, verify physiotherapists, approve or reject documents, suspend accounts, monitor bookings, review payments, resolve disputes, view operational analytics, and manage support cases.

Admin access should be protected by stricter authentication rules and audit logs.

---

9. Core Product Modules

9.1 Authentication and Account Management

The authentication module manages registration, login, session handling, logout, password recovery, role detection, protected routes, and account status checks.

Users must be assigned one primary role at registration or onboarding. The supported roles are patient, physiotherapist, and admin. Admin accounts should not be created through public registration.

9.2 Patient Profile Module

The patient profile module stores personal details needed to support booking and care coordination. This may include full name, phone number, email, gender, date of birth, emergency contact, primary address, medical notes, mobility concerns, preferred communication method, and saved locations.

Medical notes should be handled carefully because they may contain sensitive health information.

9.3 Physiotherapist Profile Module

The physiotherapist profile module stores professional details such as full name, phone number, email, license number, professional registration details, years of experience, specialties, biography, profile photo, service areas, consultation fee, availability, average rating, verification status, and uploaded documents.

A physiotherapist must remain in pending status until reviewed by an administrator.

9.4 Verification Module

The verification module handles document submission, admin review, approval, rejection, suspension, and re-verification where necessary.

Required documents may include license certificate, national ID, professional registration certificate, practice documentation, profile photo, and any other documentation required by local operational policy.

Verification states should include pending, under_review, approved, rejected, suspended, and expired.

9.5 Service and Specialty Module

The service module defines the types of physiotherapy services available on the platform. Examples include spine therapy, stroke rehabilitation, sports injury therapy, post-surgery rehabilitation, elderly mobility support, pediatric physiotherapy, neurological rehabilitation, orthopedic rehabilitation, and general musculoskeletal therapy.

Each specialty should have a patient-friendly description, estimated session duration, base pricing rules, and provider eligibility requirements.

9.6 Booking Module

The booking module is the heart of the platform. It manages booking creation, validation, physiotherapist matching, request dispatch, acceptance, payment, session tracking, completion, cancellation, and review.

The module must clearly define booking states and prevent invalid transitions. For example, a completed booking should not return to pending, and a rejected booking should not be paid unless reassigned.

9.7 Matching Module

The matching module identifies suitable physiotherapists based on patient location, selected specialty, provider availability, verification status, distance, ratings, experience, and service area.

The MVP can use a rule-based matching approach. Advanced scoring and AI-based recommendations can be introduced later.

9.8 Payment Module

The payment module handles M-Pesa STK Push initiation, callback processing, transaction validation, payment status updates, failed payment handling, refund requests, and reconciliation.

Payment status must be tied to booking status, but should remain separately auditable.

9.9 Notification Module

The notification module sends push notifications, SMS alerts, and possibly email updates for major events such as booking creation, booking acceptance, payment initiation, payment confirmation, session reminders, cancellation, completion, review prompts, and verification updates.

9.10 Reviews and Ratings Module

The reviews module allows patients to rate physiotherapists after completed sessions. Reviews should only be available for completed bookings to prevent fake ratings.

Ratings can influence matching rankings but should not be the only ranking factor.

9.11 Admin Module

The admin module provides operational control. It includes user management, physiotherapist verification, booking oversight, payment monitoring, analytics, support management, dispute resolution, and account suspension tools.

9.12 Analytics Module

The analytics module tracks product and operational metrics such as total users, active patients, active physiotherapists, booking conversion rate, booking cancellation rate, average response time, completed sessions, payment success rate, revenue, top specialties, and service area demand.

---

10. Patient User Journey

10.1 First-Time Patient Journey

The patient opens the app, sees a splash screen, moves through onboarding, selects get started, registers or logs in, completes profile setup, selects a physiotherapy need, confirms location, views available physiotherapists or allows system matching, selects time, submits a booking request, receives confirmation, makes payment when required, tracks booking progress, receives session reminders, completes the session, and submits a review.

10.2 Returning Patient Journey

A returning patient logs in, lands on the home dashboard, views active bookings, repeats a previous service, checks booking history, books another session, or updates profile details.

10.3 Emergency or Urgent Request Journey

The patient selects urgent request, chooses the condition category, confirms current location, provides short notes, submits the request, and waits for the nearest available verified physiotherapist to accept. The system should clearly communicate that urgent physiotherapy is not a replacement for emergency medical care.

---

11. Physiotherapist User Journey

11.1 Registration and Verification Journey

The physiotherapist registers, selects provider role, completes profile details, uploads required documents, submits for review, waits for admin approval, receives verification status updates, and gains access to booking features after approval.

11.2 Booking Handling Journey

An approved physiotherapist receives a booking request, reviews patient location, service type, scheduled time, notes, and expected fee, then accepts or rejects the request. If accepted, the booking moves to the payment or confirmed stage depending on the payment rules.

11.3 Session Completion Journey

The physiotherapist arrives at the patient location, starts the session, completes the session, marks it completed, and may add session notes if the platform supports care notes. Earnings are updated after successful payment and completion rules are satisfied.

---

12. Administrator User Journey

12.1 Verification Workflow

The admin logs into the admin dashboard, opens pending physiotherapist applications, reviews submitted documents, confirms completeness, approves or rejects the application, provides rejection reasons where necessary, and triggers a notification to the physiotherapist.

12.2 Booking Oversight Workflow

The admin views active bookings, filters by status, handles escalations, checks payment state, contacts users where needed, resolves disputes, and updates internal support notes.

12.3 Payment Monitoring Workflow

The admin reviews payment attempts, confirms successful transactions, identifies failed callbacks, checks reconciliation records, and escalates unresolved payment issues.

---

13. Screen Inventory

13.1 Patient App Screens

The patient app should include splash screen, onboarding screens, get started screen, login screen, registration screen, forgot password screen, home dashboard, condition selection screen, service details screen, location selection screen, physiotherapist listing screen, physiotherapist profile screen, booking details screen, schedule selection screen, payment screen, booking tracking screen, active booking screen, notifications screen, profile screen, edit profile screen, booking history screen, review screen, help and support screen, settings screen, and legal information screen.

13.2 Physiotherapist App Screens

The physiotherapist app should include registration screen, verification submission screen, verification status screen, dashboard, availability management screen, incoming requests screen, booking details screen, active session screen, booking history screen, earnings dashboard, profile management screen, reviews screen, notifications screen, support screen, settings screen, and legal information screen.

13.3 Admin Dashboard Screens

The admin dashboard should include login screen, dashboard overview, user management, physiotherapist verification queue, physiotherapist details, booking management, payment monitoring, dispute management, support tickets, analytics, notifications management, configuration, audit logs, and admin user management.

---

14. Get Started Screen Specification

The get started screen introduces the product and should communicate the main value within seconds.

The screen should include the app logo placeholder, a clear headline, a short supporting statement, a healthcare-focused illustration, one primary call-to-action, and a secondary sign-in option.

Recommended copy:

Headline: Physiotherapy Care at Home
Supporting text: Connect with verified physiotherapists near you for professional home-based treatment, rehabilitation, and mobility support.
Primary CTA: Get Started
Secondary CTA: Already have an account? Sign In

The screen should not be crowded. It should be calm, minimal, and easy to understand.

---

15. Booking Lifecycle

15.1 Booking States

The booking system should support the following states:

- draft
- pending_match
- awaiting_provider_response
- accepted
- rejected
- payment_pending
- payment_failed
- confirmed
- in_progress
- completed
- cancelled_by_patient
- cancelled_by_physio
- cancelled_by_admin
- expired
- disputed

15.2 Recommended State Flow

A patient creates a booking request. The system validates the request and creates a booking in pending_match status. The matching service finds eligible physiotherapists and dispatches the request. Once a physiotherapist accepts, the booking moves to accepted. If payment is required before confirmation, the booking moves to payment_pending. After successful payment, it becomes confirmed. At

the scheduled time, the session can move to in_progress. After service delivery, it becomes completed. The patient can then submit a review.

15.3 Invalid State Transitions

The system should block invalid state transitions. A completed booking should not be cancelled. A rejected booking should not move directly to in_progress. A physiotherapist should not accept a booking if they are not verified. A patient should not review a booking that was not completed.

---

16. Matching Logic

16.1 MVP Matching Criteria

The MVP matching engine should consider verification status, specialty, availability, distance from patient, provider service area, rating, years of experience, and current workload.

Only approved physiotherapists should appear in patient-facing results or receive booking requests.

16.2 Ranking Formula

A simple MVP scoring model can rank physiotherapists using weighted factors:

- Distance score
- Availability score
- Specialty match score
- Rating score
- Experience score
- Response reliability score

The exact weights should be tested after real usage data becomes available.

16.3 Dispatch Strategy

The platform can either show a list of available physiotherapists or dispatch the booking to a selected group of suitable providers. For the MVP, a hybrid approach may work best: allow the patient to choose from listed providers for scheduled bookings, and use automatic dispatch for urgent requests.

---

17. Payment Flow

17.1 Payment Method

The primary payment method for the Kenyan MVP should be M-Pesa through Daraja API.

17.2 STK Push Flow

The patient initiates payment. The backend creates a payment record. The system sends an STK Push request to M-Pesa. The patient enters their M-Pesa PIN. M-Pesa sends a callback response. The backend validates the callback, updates payment status, updates booking status, and notifies the patient and physiotherapist.

17.3 Payment Statuses

Payment statuses should include initiated, pending, successful, failed, cancelled, expired, refunded, and disputed.

17.4 Payment Rules

A booking should not be marked as paid unless the payment callback is verified. The frontend should not manually control payment success. Failed payments should allow retry. Duplicate callbacks should be handled safely. Each transaction should have a unique reference.

---

18. Notification Rules

Notifications should be event-based and role-specific.

Patients should receive notifications for booking request submitted, provider accepted, payment required, payment confirmed, provider on the way, session reminder, booking cancelled, session completed, and review request.

Physiotherapists should receive notifications for new booking request, booking cancelled, payment confirmed, upcoming session reminder, verification approved, verification rejected, and earnings update.

Admins should receive notifications for new verification submissions, failed payment callbacks, disputed bookings, suspicious activity, and support escalations.

---

19. Functional Requirements

19.1 Authentication Requirements

The system must allow user registration, login, logout, password recovery, session validation, role-based navigation, protected access, and account status checks.

The system must prevent suspended users from accessing active platform functions.

19.2 Patient Requirements

The system must allow patients to create and update profiles, select physiotherapy needs, choose location, submit booking requests, view physiotherapists, track active bookings, initiate payments, receive notifications, view booking history, and submit reviews.

19.3 Physiotherapist Requirements

The system must allow physiotherapists to create profiles, submit verification documents, update availability, accept or reject bookings, manage active bookings, mark sessions as completed, view earnings, receive reviews, and update professional details subject to verification rules.

19.4 Admin Requirements

The system must allow admins to view users, verify physiotherapists, approve or reject provider profiles, suspend accounts, monitor bookings, review payments, resolve disputes, and view analytics.

---

20. Non-Functional Requirements

20.1 Performance

The app should load core screens quickly on average mobile connections. Patient booking actions should feel responsive. Real-time updates should reflect booking changes without requiring users to manually refresh.

20.2 Reliability

The system should handle failed network requests, interrupted payments, duplicate callbacks, notification delays, and unavailable providers without breaking the user flow.

20.3 Scalability

The architecture should allow the platform to grow from one city to multiple regions. Service areas, pricing rules, provider coverage, and operational dashboards should be designed with expansion in mind.

20.4 Security

The system must protect user accounts, medical notes, payment records, provider documents, location data, and admin actions. Access should be role-based, validated server-side, and auditable.

20.5 Accessibility

The app should support readable text, large touch targets, clear contrast, simple navigation, meaningful icons, error messages that are easy to understand, and flows suitable for elderly users or users under stress.

20.6 Maintainability

The codebase should use modular folders, reusable components, typed data structures, shared constants, clean state management, documented backend functions, and consistent naming.

---

21. Technical Architecture

21.1 Overview

The platform will use a mobile frontend, a Convex backend, third-party authentication, M-Pesa payments, Firebase Cloud Messaging, and Google Maps integration.

The frontend communicates with backend functions for user data, bookings, payments, notifications, reviews, and admin operations. Backend functions enforce business rules and protect sensitive operations.

21.2 High-Level Architecture

Mobile clients communicate with Convex backend functions. Convex handles real-time data, business logic, and database operations. External services support authentication, payments, notifications, maps, and messaging. Admin users access internal dashboards for operational control.

21.3 Suggested Technology Stack

Frontend: React Native
State Management: Zustand
Styling: NativeWind
Navigation: React Navigation
Backend: Convex
Database: Convex Database
Authentication: WorkOS or alternative production-ready auth provider
Payments: M-Pesa Daraja API
Push Notifications: Firebase Cloud Messaging
Maps: Google Maps API
Monitoring: Sentry and Firebase Analytics
Version Control: GitHub

---

22. Frontend Architecture

22.1 Recommended Folder Structure

```
src/
  assets/
  components/
```

```
    common/
    forms/
    booking/
    profile/
    cards/
    feedback/
  constants/
  hooks/
  navigation/
  screens/
    auth/
    patient/
    physio/
    shared/
  services/
  store/
  theme/
  types/
  utils/
```

22.2 Frontend Principles

The frontend should separate UI components from business logic. Screens should remain readable and should not contain large blocks of duplicated logic. API calls should be centralized in service files or Convex hooks. Shared types should be reused across screens. Components should be reusable and accessible.

22.3 Navigation Structure

The application should use role-based navigation. Patients should enter the patient app area after login. Physiotherapists should enter the physiotherapist app area after login. Admin users should access the admin dashboard separately.

A user's verification status should affect what they can access. For example, an unverified physiotherapist should see verification status and profile completion screens, not booking requests.

---

23. Backend Architecture

23.1 Recommended Convex Module Structure

```
convex/
  auth/
  users/
  patientProfiles/
  physioProfiles/
  verification/
```

```
  services/
  availability/
  bookings/
  matching/
  payments/
  notifications/
  reviews/
  admin/
  analytics/
  schema.ts
```

23.2 Backend Principles

The backend should enforce all critical business rules. The frontend may guide the user, but the backend must validate permissions, booking eligibility, provider verification, payment state, role access, and allowed status transitions.

Sensitive actions should be logged. Admin changes should be auditable.

---

24. Data Model

24.1 Users

```
users: {
  _id,
  fullName,
  email,
  phone,
  role,
  authProviderId,
  accountStatus,
  createdAt,
  updatedAt
}
```

24.2 Patient Profiles

```
patientProfiles: {
  _id,
  userId,
  gender,
  dateOfBirth,
  emergencyContactName,
```

```
  emergencyContactPhone,
  primaryAddress,
  savedLocations,
  medicalNotes,
  mobilityNeeds,
  preferredCommunicationMethod,
  createdAt,
  updatedAt
}
```

24.3 Physiotherapist Profiles

```
physioProfiles: {
  _id,
  userId,
  licenseNumber,
  registrationBody,
  yearsOfExperience,
  specialties,
  bio,
  profilePhotoUrl,
  serviceAreas,
  baseRate,
  ratingAverage,
  ratingCount,
  verificationStatus,
  availabilityStatus,
  createdAt,
  updatedAt
}
```

24.4 Verification Documents

```
verificationDocuments: {
  _id,
  physioProfileId,
  documentType,
  fileUrl,
  status,
  reviewedBy,
  reviewedAt,
  rejectionReason,
  createdAt
}
```

24.5 Services

```
services: {
  _id,
  name,
  description,
  category,
  estimatedDurationMinutes,
  basePrice,
  isActive,
  createdAt,
  updatedAt
}
```

24.6 Availability

```
availability: {
  _id,
  physioId,
  dayOfWeek,
  startTime,
  endTime,
  isAvailable,
  createdAt,
  updatedAt
}
```

24.7 Bookings

```
bookings: {
  _id,
  patientId,
  physioId,
  serviceId,
  bookingType,
  status,
  scheduledStartTime,
  scheduledEndTime,
  patientNotes,
  location,
  distanceKm,
  priceQuoted,
  paymentStatus,
  cancellationReason,
```

```
  createdAt,
  updatedAt
}
```

24.8 Payments

```
payments: {
  _id,
  bookingId,
  patientId,
  amount,
  method,
  status,
  providerReference,
  checkoutRequestId,
  merchantRequestId,
  phoneNumber,
  paidAt,
  createdAt,
  updatedAt
}
```

24.9 Reviews

```
reviews: {
  _id,
  bookingId,
  patientId,
  physioId,
  rating,
  comment,
  createdAt,
  updatedAt
}
```

24.10 Notifications

```
notifications: {
  _id,
  userId,
  title,
  body,
  type,
  relatedEntityType,
```

```
  relatedEntityId,
  isRead,
  createdAt
}
```

24.11 Audit Logs

```
auditLogs: {
  _id,
  actorUserId,
  action,
  entityType,
  entityId,
  before,
  after,
  createdAt
}
```

---

25. Backend Function Contracts

Since Convex functions are not traditional REST endpoints by default, documentation should describe functions by module, function name, type, access role, inputs, outputs, and business rules.

25.1 Auth Functions

createUserProfile

Type: Mutation
Access: Authenticated user
Purpose: Creates the base user profile after authentication.

Input fields should include fullName, phone, email, and role.

Business rules: role must be valid; phone must be unique where required; email must match authenticated identity; accountStatus defaults to active or pending depending on role.

getCurrentUser

Type: Query
Access: Authenticated user
Purpose: Returns the current user and role-specific profile summary.

25.2 Booking Functions

createBooking

Type: Mutation
Access: Patient
Purpose: Creates a booking request.

Input fields should include serviceId, bookingType, scheduledStartTime, location, notes, and optional preferredPhysioId.

Business rules: patient must be active; service must be active; location is required; scheduled time must be valid; booking status begins as pending_match or awaiting_provider_response.

acceptBooking

Type: Mutation
Access: Verified physiotherapist
Purpose: Allows a physiotherapist to accept a booking request.

Business rules: physiotherapist must be approved; booking must be available for acceptance; physiotherapist must not already be booked at that time.

cancelBooking

Type: Mutation
Access: Patient, physiotherapist, or admin depending on conditions
Purpose: Cancels a booking.

Business rules: cancellation reason is required; cancellation policy should determine if refund rules apply.

25.3 Payment Functions

initiateMpesaPayment

Type: Action
Access: Patient
Purpose: Starts M-Pesa STK Push payment.

Business rules: booking must exist; amount must match backend calculation; phone number must be valid; duplicate pending payments should be controlled.

handleMpesaCallback

Type: Action or HTTP endpoint
Access: External provider callback
Purpose: Receives payment result from M-Pesa.

Business rules: callback must be verified; payment record must be updated safely; duplicate callbacks must not double-confirm a transaction.

---

26. Security Requirements

26.1 Role-Based Access Control

Every protected function must check the user role. Patient functions should not be available to physiotherapists unless explicitly shared. Physiotherapist functions should require approved verification where applicable. Admin functions should be restricted to admin accounts.

26.2 Data Protection

The platform stores sensitive data such as health notes, identity documents, phone numbers, location data, and payment records. Access to this information should be minimized and logged.

26.3 Payment Security

Payment success must only be confirmed through trusted backend validation, not frontend state. Transaction references should be unique. Callback handling should be idempotent.

26.4 Admin Security

Admin actions should be logged with actor, action, affected entity, timestamp, and before/after values where possible. Admin accounts should use stronger authentication controls.

26.5 Input Validation

All backend functions should validate required fields, data types, allowed statuses, phone formats, dates, pricing rules, booking ownership, provider verification, and role permissions.

---

27. Compliance and Clinical Safety Notes

PhysioHome should not present itself as an emergency medical service unless the platform has the legal, clinical, and operational capability to do so. If urgent physiotherapy requests are included, the app should clearly state that life-threatening emergencies should be directed to emergency medical services.

The platform should avoid making clinical diagnoses unless qualified professionals are involved and the legal framework supports it. AI symptom assistance, if added later, should be positioned as informational guidance, not medical diagnosis.

Patient medical notes, provider verification documents, and session-related data should be treated as sensitive information.

---

28. Error Handling and Edge Cases

The system should handle cases where no physiotherapist is available, the patient location cannot be detected, the selected provider becomes unavailable, the payment fails, the M-Pesa callback is delayed, the booking expires, the physiotherapist rejects the request, the patient cancels after payment, the physiotherapist cancels near session time, notifications fail, the user loses internet connection, or an admin suspends a provider with active bookings.

Each edge case should produce a clear user message and a safe backend state.

---

29. Testing Strategy

29.1 Frontend Testing

Frontend testing should cover authentication flows, patient booking flow, physiotherapist verification flow, provider request handling, payment screen behavior, error states, empty states, navigation protection, form validation, and responsive layouts.

29.2 Backend Testing

Backend testing should cover permission checks, booking state transitions, matching logic, payment callback handling, duplicate payment prevention, provider verification rules, cancellation rules, notification creation, and admin actions.

29.3 QA Scenarios

QA should test patient registration, physiotherapist registration, profile completion, verification rejection, verification approval, booking creation, provider acceptance, provider rejection, payment success, payment failure, booking cancellation, booking completion, review submission, support escalation, and admin suspension.

29.4 Acceptance Criteria Example

A patient should not be able to book a physiotherapist who is unverified. A physiotherapist should not be able to accept two bookings for the same time slot. A booking should not become confirmed until payment is successfully validated. A patient should not be able to review a cancelled booking. An admin should be able to see the verification history for each physiotherapist.

---

30. Deployment and Environments

The product should use separate development, staging, and production environments.

Development is for local and internal testing. Staging is for QA, demo, and pre-release validation. Production is for live users.

Environment variables should be separated by environment and should include authentication keys, Convex deployment settings, M-Pesa credentials, Firebase credentials, Google Maps keys, and monitoring keys.

Secrets should not be committed to source control.

---

31. CI/CD Workflow

The recommended workflow is GitHub push, linting, type checking, automated tests, staging deployment, QA approval, production deployment, post-release monitoring, and rollback readiness.

The team should use pull requests for feature work and require code review before merging into production branches.

---

32. Git Workflow

Recommended branches:

```
main
staging
develop
feature/*
fix/*
hotfix/*
release/*
```

Feature branch examples:

```
feature/patient-booking-flow
feature/physio-verification
feature/mpesa-payment
feature/admin-dashboard
fix/booking-status-transition
hotfix/payment-callback-error
```

---

33. Monitoring and Analytics

The platform should monitor crashes, failed payments, failed notifications, API errors, booking drop-off, provider response time, payment success rate, active bookings, completed sessions, cancellation rate, and user retention.

Recommended monitoring tools include Sentry for error tracking, Firebase Crashlytics for mobile crashes, Firebase Analytics for behavioral analytics, and internal admin dashboards for operational metrics.

---

34. Success Metrics

Product success should be measured through patient registration rate, provider approval rate, first booking conversion rate, booking completion rate, average provider response time, payment success rate, repeat booking rate, average rating, cancellation rate, patient satisfaction, provider satisfaction, and monthly gross transaction value.

Operational success should be measured through verification turnaround time, dispute resolution time, failed payment resolution time, support response time, and percentage of active verified providers.

---

35. Risks and Mitigation

35.1 Trust Risk

Patients may hesitate to invite healthcare providers into their homes. Mitigation includes strict verification, visible provider profiles, ratings, support channels, and transparent booking records.

35.2 Provider Quality Risk

Unqualified or unreliable providers could damage the platform. Mitigation includes document verification, admin review, ratings, suspension rules, and performance monitoring.

35.3 Payment Risk

M-Pesa callbacks may fail or delay. Mitigation includes idempotent callbacks, transaction reconciliation, retry logic, and admin payment monitoring.

35.4 Operational Risk

Home visits require coordination. Mitigation includes location accuracy, provider service areas, reminders, status updates, and support escalation.

35.5 Compliance Risk

Healthcare platforms handle sensitive data and clinical workflows. Mitigation includes careful data handling, access control, legal review, and clear disclaimers.

---

36. Future Roadmap

Phase 1: MVP

Build authentication, patient profiles, physiotherapist profiles, verification, booking, matching, M-Pesa payment, notifications, reviews, and admin basics.

Phase 2: Operational Strengthening

Add support tickets, better analytics, provider performance scoring, improved cancellation policies, refund support, and service-area management.

Phase 3: Care Experience Expansion

Add rehabilitation plans, progress tracking, multi-session packages, caregiver accounts, follow-up reminders, and session notes.

Phase 4: Advanced Features

Add AI symptom guidance, telehealth consultations, insurance integrations, wearable device data, corporate wellness packages, and advanced provider recommendations.

---

37. Documentation Still Needed

The following separate documents should be created after this main documentation:

1. Brand guideline document
2. UI/UX design specification
3. Wireframe documentation
4. API/function reference document
5. Database schema document
6. Admin operations manual
7. QA test case document
8. Deployment and DevOps manual
9. Privacy policy and terms of service
10. Provider verification policy
11. Payment and refund policy
12. Clinical safety and disclaimer document

---

38. Final Product Direction

PhysioHome should be built as a trust-first healthcare marketplace. The app should not only connect patients and physiotherapists; it should reduce anxiety, simplify access, and create confidence at every step.

The MVP should remain focused. The most important early priorities are verification, booking reliability, payment safety, clear communication, and simple user flows.

A successful version of PhysioHome will feel professional without being intimidating, modern without being complicated, and accessible without feeling basic.

---

39. Engineering Principles

The engineering team should build reusable components, keep business rules on the backend, validate all sensitive actions, avoid hardcoded configuration, use typed data structures, document important functions, protect role-based routes, handle edge cases, and keep the system modular.

The team should avoid overengineering the MVP, mixing business logic into UI components, allowing frontend-controlled payment status, exposing unverified providers to patients, ignoring failed payment callbacks, and building admin functions without audit logs.

---

40. Conclusion

This documentation provides a professional foundation for planning, designing, building, testing, and scaling the PhysioHome application. It improves the earlier outline by adding clearer scope, user journeys, role permissions, functional requirements, non-functional requirements, backend function contracts, booking state rules, payment rules, edge cases, security expectations, operational workflows, testing expectations, and future documentation needs.

Brand identity and visual guidelines should be created separately so that product documentation remains focused on how the app works, what it must support, and how the system should be built.